

Experiment-6:

Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

Code :

```
# =====  
# Naive Bayes Classification using Kaggle Dataset  
# =====  
  
# Upload dataset  
from google.colab import files  
uploaded = files.upload()  
  
import pandas as pd  
import io  
  
# Replace with your uploaded CSV filename if different  
filename = list(uploaded.keys())[0]  
data = pd.read_csv(io.BytesIO(uploaded[filename]))  
  
print("First 5 Rows:")  
print(data.head())  
  
# Remove unnecessary column if present  
if 'id' in data.columns:  
    data = data.drop('id', axis=1)  
  
if 'Unnamed: 32' in data.columns:  
    data = data.drop('Unnamed: 32', axis=1)  
  
# Convert diagnosis column into numbers
```

```
# M = Malignant = 1
# B = Benign = 0
data['diagnosis'] = data['diagnosis'].map({'M':1, 'B':0})
```

```
# Features and Target
X = data.drop('diagnosis', axis=1)
y = data['diagnosis']
```

```
# Train-Test Split
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# Feature Scaling
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Naive Bayes Model
from sklearn.naive_bayes import GaussianNB
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:")
print(round(accuracy*100,2),"%")

# Confusion Matrix
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
from sklearn.metrics import classification_report

print("\nClassification Report:")
print(classification_report(y_test,y_pred))

# Plot Confusion Matrix
import matplotlib.pyplot as plt
```

```
import seaborn as sns

plt.figure(figsize=(6,5))

sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=['Benign','Malignant'],
            yticklabels=['Benign','Malignant'])

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Output:

...

Choose Files
breast-cancer[1].csv

breast-cancer[1].csv(text/csv) - 124571 bytes, last modified: 8/4/2026 - 100% done
Saving breast-cancer[1].csv to breast-cancer[1].csv
First 5 Rows:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave	points_mean	\
0	0.11840	0.27760	0.3001		0.14710	
1	0.08474	0.07864	0.0869		0.07017	
2	0.10960	0.15990	0.1974		0.12790	
3	0.14250	0.28390	0.2414		0.10520	
4	0.10030	0.13280	0.1980		0.10430	

	...	radius_worst	texture_worst	perimeter_worst	area_worst	\
0	...	25.38	17.33	184.60	2019.0	
1	...	24.99	23.41	158.80	1956.0	
2	...	23.57	25.53	152.50	1709.0	
3	...	14.91	26.50	98.87	567.7	
4	...	22.54	16.67	152.20	1575.0	

	smoothness_worst	compactness_worst	concavity_worst	concave	points_worst	\
0	0.1622	0.6656	0.7119		0.2654	
1	0.1238	0.1866	0.2416		0.1860	
2	0.1444	0.4245	0.4504		0.2430	

```
... 2      0.1444      0.4245      0.4504      0.2430
     3      0.2098      0.8663      0.6869      0.2575
     4      0.1374      0.2050      0.4000      0.1625
```

```
      symmetry_worst  fractal_dimension_worst
0      0.4601      0.11890
1      0.2750      0.08902
2      0.3613      0.08758
3      0.6638      0.17300
4      0.2364      0.07678
```

```
[5 rows x 32 columns]
```

```
Accuracy:
96.49 %
```

```
Confusion Matrix:
```

```
[[70  1]
 [ 3 40]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.96	0.99	0.97	71
1	0.98	0.93	0.95	43
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

